Sorting, Custom Comparators & Greedy Basics

Put things in the right order — then take the best choice each step

Learning goals

- Know what sorting costs and when you can beat O(n log n)
- Write correct custom comparators (strict weak ordering!)
- Sort indices, pairs, and structs; use sorting to group and to dedupe
- Understand the **greedy** paradigm and its risks
- Prove a greedy with an **exchange argument**
- Solve classic greedy problems: interval scheduling, pairing, deadlines, medians

Motivation: CSES 1629 — Movie Festival

n movies, each with start a_i and end b_i. Watch the maximum number of **non-overlapping** movies. n ≤ 2·10^5.

- 2^n subsets — impossible
- Greedy idea: always pick the movie that **ends first**
- Sort by end time, scan once: O(n log n)

Why is that correct? We'll prove it.

Sorting: what it costs

- Comparison sorting needs $\Omega(n \log n)$ comparisons (decision-tree argument: $\log_2(n!) \approx n \log_2 n$)
- `std::sort`: introsort, O(n log n) worst case, not stable
- `std::stable_sort`: merge sort, O(n log n), keeps equal elements in input order
- Small value range ($0 \le a_i < K$): **counting sort** O(n + K)

```
vector<int> cnt(K, 0);
for (int x : a) cnt[x]++;
```

Sorting compound data

```
vector<pair<int,int>> v;   // sorts by first, then second
sort(v.begin(), v.end());

struct Movie { int start, end; };
vector<Movie> m(n);
sort(m.begin(), m.end(), [](const Movie &x, const Movie &y) {
    return x.end < y.end;
});
```

Trick: store `{key, original_index}` pairs to remember where things came from.

Sorting indices (argsort)

```
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0);
sort(idx.begin(), idx.end(), [&](int i, int j) {
    return a[i] < a[j];
});
// idx[0] is the position of the smallest element
```

Useful when several arrays describe the same items, or answers must be printed in input order.

Comparators: the rules

`cmp(x, y)` returns true iff x must come **strictly before** y.

- **Irreflexive:** cmp(x, x) is false → never use `<=`
- **Transitive:** cmp(x,y) and cmp(y,z) ⇒ cmp(x,z)
- **Equivalence is transitive:** "neither before the other" must be consistent

Breaking these is **undefined behavior**: `std::sort` can crash (RE) or loop.

Comparator examples

```
// descending
sort(all(a), greater<int>());
// by absolute value, ties by value
sort(all(a), [](int x, int y) {
    if (abs(x) != abs(y)) return abs(x) < abs(y);
    return x < y;
});
// multiple keys: tie() compares lexicographically
sort(all(p), [](const P &x, const P &y) {
    return tie(x.score, y.name) > tie(y.score, x.name);
}); // score desc, then name asc
```

A clever comparator: largest concatenation

Arrange numbers to form the largest number: {3, 30, 34, 5, 9} → "9534330"

- Not numeric order (30 vs 3?) and not plain string order
- Compare by concatenation: x before y iff `x + y > y + x` (as strings)
- This *is* a strict weak ordering (provable), so `sort` works

```
sort(all(s), [](const string &x, const string &y) {
    return x + y > y + x;
});
```

Greedy algorithms

> Build the solution step by step, always making the choice that looks best **now**, never undoing it.

- Pros: simple, fast (usually sort + scan, O(n log n))
- Cons: often **wrong**! Locally best ≠ globally best
- A greedy needs a **proof** (or at least a stress test vs. brute force)

Greedy can fail: coin change

Coins {1, 3, 4}, make 6 with fewest coins.

- Greedy (largest first): 4 + 1 + 1 → 3 coins
- Optimal: 3 + 3 → 2 coins

Greedy works for "canonical" systems like {1, 2, 5, 10, ...} but not in general → **DP** (Week 14).

Always look for a small counterexample before trusting a greedy.

Exchange argument (proof template)

1. Let G be the greedy solution and O any optimal solution
2. Find the first place they differ
3. Show you can **swap** O's choice for G's choice without making O worse
4. Repeat → O becomes G, so G is optimal

If the swap can make things worse, the greedy is likely wrong.

Proof: Movie Festival (earliest end first)

- Let g = movie with smallest end time. Take an optimal set O, its first movie o
- end(g) ≤ end(o), so replacing o by g conflicts with nothing else in O
- Still |O| movies, now starting with g ✓
- Remove everything overlapping g, repeat on the rest

```
o:    |-------|
g:  |----|           (ends earlier → more room for the rest)
rest:        |--| |---|
```

Movie Festival — code

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<pair<int,int>> m(n);          // {end, start}
    for (auto &[e, s] : m) cin >> s >> e;
    sort(m.begin(), m.end());
    int cnt = 0, lastEnd = 0;            // times are >= 1
    for (auto [e, s] : m)
        if (s >= lastEnd) { cnt++; lastEnd = e; }
    cout << cnt << '\n';
}
```

Sorting by start time or by length is **wrong** — find counterexamples!

CSES 1090 — Ferris Wheel

Children weights p_i, gondola capacity x, ≤ 2 children per gondola. Minimize gondolas.

- Sort. Pair the **heaviest** remaining with the **lightest** if they fit; else heaviest goes alone
- Exchange: if the heaviest can share with anyone, sharing with the lightest is never worse

```
sort(all(p));
int i = 0, j = n - 1, ans = 0;
while (i <= j) {
    if (i < j && p[i] + p[j] <= x) i++;
    j--; ans++;
}
```

CSES 1074 — Stick Lengths

Make all sticks equal length; cost = Σ |p_i − L|. Minimize.

- Optimal L is the **median** of the sorted array
- Why: moving L past the median increases distance to more than half the points

```
sort(all(p));
long long L = p[n / 2], cost = 0;
for (long long x : p) cost += llabs(x - L);
```

CSES 1630 — Tasks and Deadlines

Tasks with duration a_i, deadline d_i. Reward for task = d_i − finish time (can be negative). Do all tasks; maximize total reward.

- Total = Σ d_i − Σ finish_i. Σ d_i is fixed!
- Minimize sum of finish times → **shortest job first**
- Exchange: swapping adjacent (long, short) to (short, long) decreases the sum

Scheduling with deadlines: ABC131 D — Megalomania

Jobs with duration A_i and deadline B_i; one at a time from time 0. Can all finish on time?

- Sort by deadline (**Earliest Deadline First**), simulate, check each finish ≤ deadline
- Exchange: if a job with a later deadline runs right before one with an earlier deadline, swapping them never creates a new violation

Sorting as a preprocessing tool

- **Grouping:** after sorting, equal items are adjacent → count runs
- **Dedup:** `sort` + `unique`
- **Closest pair in 1D:** answer is among adjacent elements after sorting
- **Sweep:** sort events by time, process in order (CSES 1619 Restaurant Customers)

```
// events: +1 at arrival, -1 at leaving
vector<pair<int,int>> ev;
for (...) { ev.push_back({a, +1}); ev.push_back({b, -1}); }
sort(all(ev));
int cur = 0, best = 0;
for (auto [t, d] : ev) best = max(best, cur += d);
```

Complexity summary

Problem	Algorithm	Time
Movie Festival	sort by end, scan	O(n log n)
Ferris Wheel	sort, two ends	O(n log n)
Stick Lengths	sort, median	O(n log n) (O(n) with `nth_element`)
Tasks & Deadlines	sort by duration	O(n log n)
Restaurant Customers	sort events, sweep	O(n log n)

Pitfalls

- Comparator with `<=` → undefined behavior, possible RE
- Sorting by the "obvious" key without a proof (start time, length, ratio...)
- Ties: does the order of equal keys matter? (Restaurant: departures before arrivals at equal times?)
- Overflow in sums of finish times → `long long`
- Comparator copying big objects → take `const T&`

Practice problems

- CSES 1629 Movie Festival · 1090 Ferris Wheel · 1074 Stick Lengths · 1619 Restaurant Customers · 1630 Tasks and Deadlines · 1084 Apartments
- AtCoder ABC131 D Megalomania · ABC085 B Kagami Mochi
- Codeforces 230A Dragons · 451B Sort the Array · 1353B Two Arrays And Swaps · 545D Queue · 1029C Maximal Intersection
- Kattis `classrooms`

Summary

- Sorting is O(n log n) and unlocks greedy, sweeps, grouping, and binary search
- Comparators: strict, transitive, never `<=`
- Greedy = choose locally best; **prove** it with an exchange argument or stress test it
- Classic greedy keys: earliest end, heaviest+lightest, median, shortest job, earliest deadline
- Next week: **binary search** — searching sorted data and searching the answer